

SUDARSHAN UAV

Autonomous Quadcopter — Project Report & Onboarding Guide

Version 1.2 · SUDARSHAN-RADHA Project

Generated 03 June 2026

FLIGHT CONTROLLER
Arduino Mega 2560

WIFI BRIDGE
ESP32 DevKit

GROUND STATION
Python / Web GCS

MOTOR DRIVER
Arduino Uno + PCA9685

TABLE OF CONTENTS

#	Section	Page
1	Project Overview	3
2	Hardware Components	4
3	System Architecture	5
4	Communication Protocols	6
5	Wiring Reference	7
6	Flight Modes	8
7	Command Protocol	9
8	PID Configuration & Tuning	11
9	Dead-Man Switch (3-Layer Safety)	12
10	Onboarding — First-Time Setup	13
11	Running the GCS	15
12	Technical Debt Audit & Fixes (v1.2)	16
13	Known Limitations	19
14	Test & Validation	20
15	Quick-Start Reference Card	21

1. PROJECT OVERVIEW

SUDARSHAN UAV is an autonomous quadcopter development project. The system is built around off-the-shelf Arduino and ESP32 hardware, aiming for a self-contained flying platform with full ground-station control, dead-man safety switches, altitude hold, and mission-preset autonomous flight — all without requiring a dedicated flight-controller board.

Project Name	SUDARSHAN UAV (RADHA Project)
Version	1.2
Branch	claude/repo-analysis-risk-plan-W1XIB
Developer	Sudarshan
Status	Active development — lab-ready, pre-flight validation pending
GCS	Python/Tkinter laptop GCS + ESP32 Web GCS (phone browser)
Frame	X-frame quadcopter, 4× brushless 1900KV motors
Battery	3S LiPo (warn: 10.5 V cut-off: 9.9 V full: 12.6 V)
Control Rate	250 Hz (Mega FC) Telemetry: 10 Hz GPS forward: 5 Hz

2. HARDWARE COMPONENTS

Component	Part / Model	Role
Flight Controller	Arduino Mega 2560	250 Hz control loop, 4 PID loops, 8 flight modes, UART hub
Motor Driver	Arduino Uno / Nano	Receives 10-byte binary packets; drives PCA9685 → 4 ESCs
WiFi Bridge	ESP32 DevKit (30-pin)	WiFi AP at 192.168.4.1; bridges GCS ↔ FC; web GCS server
IMU	MPU6050 (I2C 0x68/69)	6-axis accel+gyro; complementary filter for roll/pitch/yaw
Sonar	HC-SR04	Altitude measurement (25 Hz); landing detection
PWM Driver	PCA9685 (I2C 0x40)	16-ch I2C → 4 ESC PWM signals at 50 Hz
Motors	4× Brushless 1900KV	X-frame: FL■ FR■ RL■ RR■ (CW/CCW alternating)
ESCs	4× BLHeli/SimonK-compatible	PWM 1000–1950 μs; calibrated via CAL_ESC command
Battery	3S LiPo 11.1 V	Voltage divider on A0 (47kΩ + 10kΩ); auto-LAND at 9.9 V
Ground Station	Laptop (Python 3.8+)	Full GCS UI: telemetry, presets, preflight, nav, guide tabs
Mobile GCS	Any phone browser	ESP32 web GCS — no laptop required; WS push at 10 Hz
GPS	Phone GPS app (NMEA)	NMEA over TCP :5762 → ESP32 → FC at 5 Hz

Note on ESP32 voltage: The ESP32 GPIO is 3.3 V. The Mega TX (5 V) must go through a voltage divider (e.g. 1kΩ + 2kΩ) before reaching ESP32 RX. The Mega RX can accept 3.3 V directly.

3. SYSTEM ARCHITECTURE

The system is a five-layer stack. Each layer has a single responsibility and communicates only with its immediate neighbours. The ESP32 is a transparent relay — it has no flight logic. The FC can land itself autonomously even if all external comms fail.

Layer	Device	Connects to	Protocol
1 — Operator UI	Laptop (Python GCS) or Phone browser	ESP32 via WiFi	TCP :5760 JSON or WS :81
2 — WiFi Bridge	ESP32	GCS (TCP) Phone GPS (TCP) Mega (UART)	TCP :5760/:5762 UART 115200
3 — Flight Control	Arduino Mega 2560	ESP32 (UART) Uno (UART) MPU6050 (I2C) Sonar (GPIO)	JSON Binary 10-byte I2C GPIO
4 — Motor Driver	Arduino Uno	Mega (UART) PCA9685 (I2C)	Binary 10-byte I2C
5 — Actuators	PCA9685 + 4 ESCs + 4 Motors	Uno (I2C) Battery	PWM 50 Hz

Data Flow Summary

```
GCS command → ESP32 TCP → Mega UART (JSON) → PID compute → Uno UART (10-byte) → PCA9685 → ESC PWM
FC telemetry → Mega UART → ESP32 → GCS TCP / WS push (10 Hz)
Phone GPS → TCP :5762 → ESP32 parser → Mega UART GPS cmd (5 Hz)
```

Timing Hierarchy

Rate	Task	Notes
250 Hz	Mega control loop	IMU read → complementary filter → 4× PID → motor mix → Uno packet
~25 Hz	HC-SR04 sonar read	pulseIn() outside 250 Hz loop (blocking up to 23 ms)
10 Hz	Telemetry JSON to ESP32	Every 25th control-loop tick
5 Hz	GPS command from ESP32 to FC	200 ms interval; ESP32 throttles phone data
1 Hz	GCS PING heartbeat	Resets DMS on both ESP32 and FC
0.04 Hz	Battery ADC sample	Once per telemetry frame

4. COMMUNICATION PROTOCOLS

4.1 GCS ↔ ESP32 (TCP :5760)

Newline-delimited UTF-8 JSON. Each message is one JSON object followed by `\n`. Optional AES-128-CBC encryption: wire format is `base64(IV + AES_CBC(PKCS7(json))) + \n`. A random 16-byte IV is prepended to every message.

4.2 ESP32 ↔ Mega FC (UART 115200)

Same newline-delimited JSON format as GCS↔ESP32. The ESP32 is a transparent relay — it forwards commands from GCS to FC and telemetry from FC to GCS without modification.

4.3 Mega FC → Uno Motor Driver (UART 115200, 250 Hz)

10-byte binary packet, little-endian start byte, big-endian motor values:

```
[ 0xAA ] [ fL_H fL_L ] [ fr_H fr_L ] [ rL_H rL_L ] [ rr_H rr_L ] [ XOR ]
```

Start byte: 0xAA. Motor values: 16-bit big-endian μs (range 1000–1950). XOR checksum: XOR of bytes 1–8. Corrupt packets are silently discarded by the Uno.

4.4 Telemetry JSON (FC → GCS, 10 Hz)

```
{"roll":1.2, "pitch":-0.5, "yaw":182.3, "alt_cm":45, "bat_mv":11800, "mode":"HOVER", "armed":1, "imu_ok":1, "sonar_ok":1}
```

4.5 GPS JSON (ESP32 → FC, 5 Hz)

```
{"cmd":"GPS", "lat":28.6139, "lon":77.2090, "alt":215.0, "heading":182.3, "baro_cm":21500, "fix":1, "sats":8}
```

4.6 WebSocket (ESP32 port 81 → Phone Browser)

ESP32 pushes merged telemetry+GPS JSON over WebSocket at FC rate (~10 Hz). The phone browser uses WS as primary channel and falls back to HTTP GET `/api/telem` (1 s poll) if WS disconnects. Auto-reconnect every 3 s.

5. WIRING REFERENCE

5.1 Mega 2560 Pin Assignments

Mega Pin	Signal	Connected To	Notes
Pin 18 (TX1)	UART TX → ESP32	ESP32 GPIO16 (RX2) via divider	5V→3.3V: 1kΩ+2kΩ divider
Pin 19 (RX1)	UART RX ← ESP32	ESP32 GPIO17 (TX2)	3.3V direct — Mega RX tolerates it
Pin 16 (TX2)	Motor packet TX	Uno Pin 0 (RX)	115200 baud; disconnect when flashing Uno
Pin 17 (RX2)	Motor ready RX	Uno Pin 1 (TX)	Uno sends 'READY' after ESC arming
Pin 20 (SDA)	I2C SDA	MPU6050 SDA	4.7kΩ pull-up to 3.3V
Pin 21 (SCL)	I2C SCL	MPU6050 SCL	4.7kΩ pull-up to 3.3V
Pin 7	Sonar TRIG	HC-SR04 TRIG	5V GPIO output
Pin 8	Sonar ECHO	HC-SR04 ECHO via divider	5V→3.3V needed if using 3.3V Mega
A0	Battery ADC	LiPo + via 47kΩ+10kΩ	Scale: 5000/1023 × 5.7
GND	Common ground	ESP32 GND, Uno GND, sensors	CRITICAL — must be common

5.2 Uno / PCA9685 / ESC Connections

Uno Pin	Signal	Connected To	Notes
Pin 0 (RX)	Packet RX	Mega Pin 16 (TX2)	■ Disconnect when flashing via USB
Pin 1 (TX)	Ready TX	Mega Pin 17 (RX2)	■ Disconnect when flashing via USB
A4 (SDA)	I2C SDA	PCA9685 SDA	Address 0x40 default
A5 (SCL)	I2C SCL	PCA9685 SCL	
5V	PCA9685 VCC	PCA9685 VCC pin	
GND	Common GND	PCA9685 GND	

5.3 PCA9685 ESC Channel Mapping

PCA9685 Channel	Motor Position	Rotation	Mix sign (roll/pitch/yaw)
CH 0	Front-Left (FL)	Clockwise ■	+pitch +roll -yaw
CH 1	Front-Right (FR)	Counter-CW ■	+pitch -roll +yaw
CH 2	Rear-Left (RL)	Counter-CW ■	-pitch +roll +yaw
CH 3	Rear-Right (RR)	Clockwise ■	-pitch -roll -yaw

Motor layout (top view X-frame): FL(■) — FR(■) diagonally opposite. Props: CW motors use CW props, CCW motors use CCW props.

6. FLIGHT MODES

Mode	Value	Description	Entry / Exit
DISARMED	0	Motors off. Safe state.	Boot default; DISARM cmd; landing detect
HOVER	1	Level hold + altitude hold. Roll/pitch/yaw setpoints zeroed.	DISARM cmd (→HOVER); HOVER cmd from any mode
LAND	2	Slow descent at 12 cm/s. Auto-disarm at alt < 8 cm (×6).	LAND cmd; battery critical; FAILSAFE
RTL	3	■ NOT IMPLEMENTED — falls back to LAND immediately	RTL cmd (→LAND)
OVERRIDE	4	Direct roll/pitch/yaw/throttle setpoints from GCS.	OVERRIDE cmd with roll/pitch/yaw/thr
FAILSAFE	5	Same as LAND. Triggered by 30 s DMS timeout.	FC DMS fires after 30 s silence
KILL	6	Instant motor cut. No recovery — must power-cycle.	KILL cmd (double-tap on web GCS)
IMUMISSION	7	IMU-only autonomous preset path execution.	PRESET cmd with segments[]

State Machine Summary

[illegible]

7. COMMAND PROTOCOL

All commands are sent as JSON objects with a **cmd** key, terminated by `\n`. The FC responds with an ACK JSON: `{"ack":"CMD", "status":"OK"}` or `{"ack":"CMD", "status":"ERR", "msg":"reason"}`.

Command	JSON Payload	Armed?	Description
ARM	<code>{"cmd":"ARM"}</code>	No	Arm ESCs (2.5 s), save home GPS, reset PIDs → HOVER
FORCE_ARM	<code>{"cmd":"FORCE_ARM"}</code>	No	ARM bypassing IMU check (admin only)
DISARM	<code>{"cmd":"DISARM"}</code>	Any	Immediate motor cut → DISARMED
HOVER	<code>{"cmd":"HOVER"}</code>	Yes	Reset setpoints, hold current altitude
LAND	<code>{"cmd":"LAND"}</code>	Yes	Slow descent, auto-disarm at ground
KILL	<code>{"cmd":"KILL"}</code>	Yes	Emergency motor cut (no recovery)
PING	<code>{"cmd":"PING"}</code>	Any	DMS heartbeat — no response
OVERRIDE	<code>{"cmd":"OVERRIDE", "roll":5, "pitch":-3, "yaw":0, "throttle":1180}</code>	Yes	Direct attitude + throttle setpoints
ALT_HOLD	<code>{"cmd":"ALT_HOLD", "alt_cm":150}</code>	Yes	Update altitude PID target (30–500 cm)
PRESET	<code>{"cmd":"PRESET", "segments":[{"bearing":0, "dist_m":5, "speed":0.5}]}</code>	Yes	Upload & execute up to 16 waypoint segments
MOTOR_TEST	<code>{"cmd":"MOTOR_TEST", "motor":"FL", "throttle":1100, "duration_ms":1500}</code>	No	Spin one motor for bench test (props off!)
SPIN_CH	<code>{"cmd":"SPIN_CH", "ch":0, "thr":1100, "dur":2000}</code>	No	Spin raw PCA9685 channel (motor-ID wizard)
SET_MOTOR_MAP	<code>{"cmd":"SET_MOTOR_MAP", "fl":2, "fr":0, "rl":3, "rr":1}</code>	No	Save channel→motor mapping from wizard
CAL_ESC	<code>{"cmd":"CAL_ESC"}</code>	No	ESC throttle range calibration (props off!)
GPS	<code>{"cmd":"GPS", "lat":28.6, "lon":77.2, "fix":1, "sats":8, ...}</code>	Any	Inject GPS data from ESP32

Web GCS priority lock: When the Python laptop GCS is connected on TCP :5760, the ESP32 blocks all web GCS commands (returns `{ok:0, locked:1}`). Override codes: **1410** (session unlock) or **980752** (master — unlocks everything).

8. PID CONFIGURATION & TUNING

Axis	Kp	Ki	Kd	Output Limit	Notes
Roll	1.8	0.05	0.80	±300 µs	Negative = left bank. Flip sign if drone rolls wrong way.
Pitch	1.8	0.05	0.80	±300 µs	Negative = nose down.
Yaw	2.0	0.02	0.00	±120 µs	Kd=0: gyro noise would saturate derivative on yaw.
Altitude	3.0	0.10	1.50	±250 µs	Integrator frozen when sonar is stale (v1.2 fix).

Tuning Guide (Ziegler-Nichols-lite for beginners)

- Step 1: Set Ki=0, Kd=0. Increase Kp until the drone oscillates at hover (Ku).
- Step 2: Set Kp = 0.6 × Ku as starting point.
- Step 3: Add Kd slowly until oscillations damp within 1–2 cycles.
- Step 4: Add Ki slowly to correct steady-state altitude/heading error.
- Step 5: For yaw: leave Kd=0; gyro noise is too high for derivative.
- Step 6: Re-tune after changing props, battery, or payload weight.

IMU yaw sign: If the drone spins the wrong direction on first flight, change **IMU_YAW_SIGN** from 1 to -1 in SUDARSHAN_FC.ino. Do this on the bench, NOT in the air.

Complementary filter: CF_ALPHA=0.98 is tuned for 250 Hz. The Mega caps loop dt at 10 ms to prevent gyro spikes. If the drone rocks at ~1 Hz during hover, increase CF_ALPHA slightly (0.985).

9. DEAD-MAN SWITCH — 3-LAYER SAFETY

Three independent watchdogs ensure the drone enters a safe state even if GCS software crashes, WiFi drops, or the ESP32 locks up. Each layer acts independently.

Layer	Lives In	Timeout	Trigger Condition	Action	Back
1 — GCS DMS	Python GCS (laptop)	30 s	Operator idle — no button/slider moved	Sends HOVER cmd → ESP32	Operator inattenti
2 — ESP32 DMS	ESP32 firmware	30 s	No GCS TCP packet received	Injects HOVER directly into FC	GCS crash / Wi-Fi
3 — FC DMS	Mega 2560 firmware	30 s	No UART command received	Mode → FAILSAFE → LAND	ADISARM fail

DMS Reset Path

```
Operator clicks/moves GCS → GCS sends PING every 25 s (+ every command) PING → ESP32 lastPing
reset → FC lastCmdMs reset Web GCS: JS sends PING every 25 s via POST /api/cmd while tab is open
```

Web-only operation (no laptop): The ESP32 DMS stays disarmed until a TCP GCS connects. The FC DMS is always active. The web GCS JavaScript sends a PING every 25 s automatically, keeping the FC DMS alive as long as the phone browser tab is open.

10. ONBOARDING — FIRST-TIME SETUP

10.1 Required Libraries (Arduino IDE)

Library	Install via	Used By
ArduinoJson v6	Library Manager	Mega FC, ESP32
Adafruit PWM Servo Driver	Library Manager	Uno motor driver
WebSockets (Markus Sattler)	Library Manager	ESP32 bridge
Wire.h	Built-in	All boards

10.2 Flashing Sequence

Step 1: Flash Uno first

Open SUDARSHAN_MOTOR_UNO.ino. Disconnect wires from Pin 0 and Pin 1. Select Board: Arduino Uno. Upload. Reconnect wires after flashing.

Step 2: Flash Mega FC

Open SUDARSHAN_FC.ino. Select Board: Arduino Mega 2560, Port: COMx/ttyUSBx. Upload.

Step 3: Configure ESP32 credentials

Copy credentials.h.example to credentials.h. Edit WIFI_SSID, WIFI_PASS, and optionally AES_KEY. The SSID is the AP the ESP32 broadcasts — default SUDARSHAN_AP / radha2026.

Step 4: Flash ESP32

Open ATLAS_ESP32_Bridge_v2.ino in Arduino IDE. Select Board: ESP32 Dev Module. Select correct port. Upload.

Step 5: Python GCS setup

cd SUDARSHAN-RADHA/GCS/ then pip install cryptography. Copy credentials.py from template. Set ENCRYPT_ENABLED=True if using AES.

Step 6: Power-on sequence

Power Mega+Uno first (wait for ESC arming beeps ~3 s). Then power ESP32. Connect phone/laptop to WiFi SUDARSHAN_AP. Run: python3 radha_gcs.py

10.3 Common Boot Messages (USB Serial Monitor)

```
■■ SUDARSHAN FC v1.0 ■■ [UNO ] waiting for motor driver (ESC arming ~3s)... [UNO ] motor driver
ready — ESCs armed [IMU ] MPU6050 OK at 0x68 [IMU ] Calibrating — keep still... [IMU ] Offsets:
0.123 / -0.045 / 0.891 [SONAR] 45.2 cm [PID ] initialized [READY] DISARMED — waiting for GCS
```

If IMU not found: Check SDA→Pin20, SCL→Pin21, VCC→3.3V, GND common. ARM will be blocked until MPU6050 is detected.

11. RUNNING THE GCS

11.1 Python Laptop GCS

- Connect laptop to WiFi: **SUDARSHAN_AP** (password: radha2026)
- Run: **python3 radha_gcs.py**
- Login dialog appears — enter credentials from auth.json
- Click **CONNECT** — status bar turns green on success
- Run **PREFLIGHT** tab tests (IMU, sonar, motor direction — props OFF)
- Once critical tests pass, ARM button unlocks
- Optionally click **INAUGURATE** for ceremonial T-minus countdown
- Click **ARM** → drone enters HOVER mode
- Monitor telemetry panel: roll/pitch/yaw/altitude/battery
- Use PRESET tab to build and execute autonomous paths

11.2 Phone Web GCS (no laptop)

- Connect phone to WiFi: **SUDARSHAN_AP**
- Open browser → navigate to **http://192.168.4.1/**
- GCS page loads — telemetry shows '---' until Mega is powered
- Tap **START GPS** → grant location permission → GPS streams automatically
- Tap **ARM** → enter priority code if laptop GCS is also connected
- KILL button: tap once (turns red/armed), tap again within 3 s → KILL sent
- DMS countdown visible in status bar — resets on every command

11.3 GCS Tabs (Python)

Tab	Purpose
FLIGHT	Manual control: ARM/DISARM/HOVER/LAND/KILL, OVERRIDE sliders, ALT setpoint
PRESET	Build autonomous paths: bearing+distance segments; click-mode canvas input
PREFLIGHT	Hardware validation: IMU, sonar, motor direction tests (requires DISARMED)
NAV	Navigation compass: target lat/lon → bearing+distance → FLY TO preset
GUIDE	Training checklist + SIM LOCK (blocks ARM in simulation context)

12. TECHNICAL DEBT AUDIT & FIXES (v1.2)

A full technical debt audit was performed against all four firmware/software files. The table below lists every finding. Items marked **FIXED** were resolved in the v1.2 commit on branch *claude/repo-analysis-risk-plan-W1XIB*.

#	Sev	Component	Issue	Status
1	CRITICAL	FC	Sonar EMA cold-start: alt=0 on ARM causes immediate descent	FIXED
2	CRITICAL	FC	Motor mix asymmetric saturation: loses attitude authority	FIXED
3	CRITICAL	Uno	UART packet desync: no timeout — stuck mid-packet forever	FIXED
4	CRITICAL	FC	PRESET div-by-zero if dist_m=0 in segment	FIXED
5	CRITICAL	FC	UART receive buffer: String heap grows without bound on bad packets	FIXED
6	HIGH	ESP32	DMS race: KILL/LAND overridden by coincident DMS fire at T=30s	FIXED
7	HIGH	ESP32	AES malloc failure silently transmits plaintext	FIXED
8	HIGH	FC	GPS compass correction unbounded: stale GPS yanks heading	FIXED
9	HIGH	Python GCS	Seq gap false-positive: ESP32 reboot floods log, loses real warnings	FIXED
10	MEDIUM	FC	Gyro spike (I2C glitch) saturates PID integrator	FIXED
11	MEDIUM	FC	Altitude PID wind-up when sonar is stale	FIXED
12	MEDIUM	FC	SET_MOTOR_MAP accepted mid-flight: violent asymmetric thrust	FIXED
13	MEDIUM	Python GCS	Preflight motor test ACK race: bad motor reports PASS	FIXED
14	MEDIUM	Python GCS	NAV FLY TO sends unrecognised JSON to FC — silently ignored	FIXED
15	LOW	FC	RTL is dead code — falls back to LAND without GPS	OPEN (hardware)
16	LOW	FC	CF filter tuned for 250 Hz; slower loops shift alpha	OPEN (hardware)
17	LOW	ESP32	TCP client/server mode doc ambiguity for GPS app	OPEN (docs)
18	LOW	All	No boot handshake/version sync between boards	OPEN
19	LOW	Python GCS	Flight log timestamps not synchronised with FC millis()	OPEN
20	LOW	ESP32	Passwords hardcoded in firmware (physical access = bypass)	OPEN (by design)

13. KNOWN LIMITATIONS

■ RTL Not Implemented

Return-to-Launch requires a dedicated GPS module on the Mega. Currently MODE_RTL immediately falls back to LAND in place. The web GCS and Python GCS expose an RTL button but the operator will see an ERR ACK and the drone will land where it is.

■ IMU-Only Navigation

PRESET missions use gyro heading and sonar altitude. There is no position hold. Wind and prop wash will cause the drone to drift off the intended path. Outdoor use without GPS requires calm conditions.

■ Sonar Range & Reliability

HC-SR04 has a practical range of ~2–3 m. Readings become unreliable above 2 m, in very dusty/humid conditions, or when the drone tilts beyond ~30°. The 'sonar stale' flag guards against this but altitude hold is lost.

■ No Position Hold

Without GPS, the drone has no way to hold a GPS position. HOVER holds attitude (level) and altitude (sonar), but it will drift laterally in any wind.

■ Single-Axis Battery Voltage

Battery voltage is read from a single analog pin via a resistor divider. The divider ratio assumes a 3S pack. Using a different cell count without updating BATT_SCALE will report incorrect voltage and may trigger premature LAND.

■ WiFi AP Only

The ESP32 runs as an Access Point only. There is no station-mode fallback. If interference prevents WiFi connection, the only option is to power-cycle the ESP32.

■ Uno Upload Requires Wire Disconnect

Pins 0 and 1 on the Uno are shared between UART (motor packets) and the USB-to-Serial chip. The wires from the Mega must be physically disconnected every time the Uno sketch is updated.

14. TEST & VALIDATION

14.1 Python Unit Tests

Run from SUDARSHAN-RADHA/GCS/:

```
pip install pytest pytest tests/ -v
```

Test File	Coverage
test_protocol_parsing.py	Telemetry JSON routing, ACK parsing, info events, seq gap detection
test_connection_manager.py	TCP send/recv, JSON framing, DMS timeout, reconnect
test_dms.py	Dead-man switch timer accuracy, reset logic, fire action

14.2 Bench Test Procedure

Full procedure in SUDARSHAN-RADHA/TEST/BENCH_TEST.md. Summary:

Test	Expected Result	Safe?
ESP32 boot	AP SUDARSHAN_AP visible in WiFi scan	Yes
GCS TCP connect	Status bar green, telemetry flowing	Yes
MPU6050 detection	[IMU] OK at 0x68 in serial monitor	Yes
Sonar sanity	alt_cm ~25–40 cm when on bench	Yes
Battery voltage	bat_mv within 200 mV of multimeter	Yes
MOTOR_TEST FL	FL motor spins briefly (props OFF!)	Props off
MOTOR_TEST all	All 4 motors confirmed spinning	Props off
Motor direction	Verify CW/CCW per wiring diagram	Props off
Phone GPS streaming	lat/lon update in GCS GPS panel	Yes
ARM → HOVER	Drone level, altitude holds	Tethered
DMS timeout	HOVER/FAILSAFE after 30s silence	Tethered
KILL command	Motors cut instantly	Tethered

14.3 Pre-Flight Checklist (Outdoor)

Full checklist in SUDARSHAN-RADHA/TEST/Pre-Flight Checklist.md

- Battery charged (≥ 11.5 V), securely mounted
- All motor screws tight, props correct direction (CW/CCW)
- Props balanced and free of cracks
- All wires routed away from props
- ESC calibration confirmed (CAL_ESC run once on new ESCs)
- Motor map verified (SET_MOTOR_MAP wizard run after any rewiring)
- GCS connected, telemetry nominal, sonar reading correct altitude
- IMU calibration: placed level, motionless during boot
- Preflight tab: all critical tests PASS
- Phone GPS fix acquired (≥ 6 satellites recommended)
- Area clear of people and obstacles within 10 m radius

- ■ Arm only after verbal 'CLEAR' call to bystanders

15. QUICK-START REFERENCE CARD

LAPTOP GCS

```
cd SUDARSHAN-RADHA/GCS/ python3 radha_gcs.py
```

WiFi: **SUDARSHAN_AP** / radha2026
IP: **192.168.4.1** | TCP :5760

Startup sequence:

- Power Mega+Uno (wait 3 s ESC beeps)
- Power ESP32
- Connect WiFi, run python3 radha_gcs.py
- Login → Connect → Preflight tab
- All critical tests PASS → ARM unlocks
- ARM → HOVER → Fly

Emergency:

- KILL button (double-tap on phone)
- DISARM button on laptop
- Kill WiFi → FC DMS fires in 30 s → LAND

PHONE WEB GCS

```
http://192.168.4.1/
```

No laptop needed. Works on any browser.

Controls:

- ARM / DISARM / HOVER / LAND
- KILL: tap once (red) → tap again in 3 s
- ALT▼: altitude setpoint slider
- PATH▼: tap waypoints on canvas
- NAV▼: fly to GPS coordinate
- START GPS: stream phone location

Priority lock:

- If laptop GCS connected → web blocked
- Enter **1410** to override (session)
- Enter **980752** to override (master)

ESC Throttle Range

Signal	µs	Meaning
ESC_ARM	1000	Arming pulse — motors off, ESC armed
ESC_MIN	1050	Minimum running throttle
ESC_IDLE	1150	Hover baseline — tune for your AUW
ESC_MAX	1950	Maximum throttle

3S LiPo Reference

Condition	Voltage	Action
Full charge	12.6 V	Normal operation
Nominal hover	11.1 V	Normal operation
WARN threshold	10.5 V	Land soon — telemetry flag set
CRIT threshold	9.9 V	Auto-LAND triggered immediately
Storage	11.4 V	Ideal for long-term storage